

§ 6.3.2 非递归遍历二叉树

Recursion **PreOrder** traverse: **root**, left child, right child;

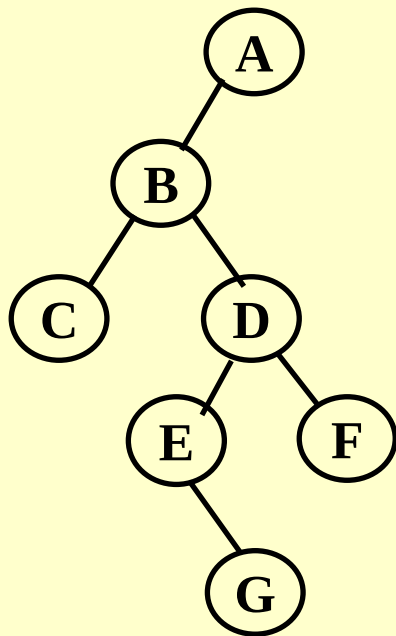
A B C D E G F

Recursion **InOrder** traverse: left child, **root**, right child;

C B E G D F A

Recursion **PostOrder** traverse: left child, right child, **root**;

C G E F D B A



144.二叉树的前序遍历

94.二叉树的中序遍历

145.二叉树的后序遍历

§ 6.3.2 非递归遍历二叉树

非递归中序遍历**InOrder**: left child, **root**, right child;

思想：自定义栈去记录递归路径，代替系统实现递归时的栈操作

For pointer p,

1) 若p非空，将p指向的结点进栈，p指向左孩子；

If p is not NULL,

Then Push(S,p); p=p->lchild;

2) 若p为空，则出栈且p指向该结点，访问结点，p指向右孩子

If p is NULL,

Then Pop(S,p); Visit(p->data);

p=p->rchild;

3) 直到p为空，栈也为空。

Until p is NULL and stack is NULL.

递归与非递归区别？

```
Status InOrderTraverse(T){
```

```
    InitStack(S); p=T;
```

```
    while (p||StackEmpty(S)){
```

```
        if(p){//当前指针不为空,进栈,遍历左
```

```
            Push(S,p);p=p->lchild;
```

```
        }
```

```
        else{//为空,从左,访问根,返回上一层
```

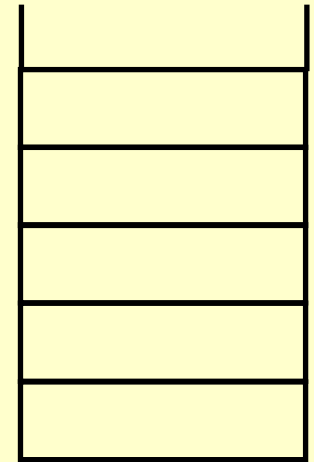
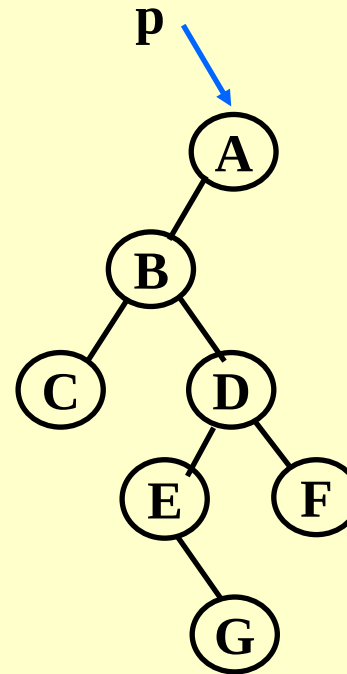
```
            Pop(S,p); //从右, 直接退栈
```

```
            printf(p->data);
```

```
            p=p->rchild;
```

```
        }
```

```
    return OK;}
```



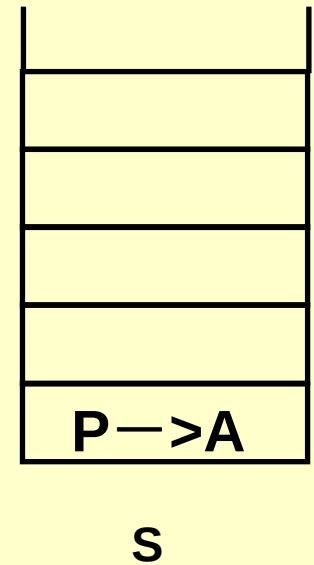
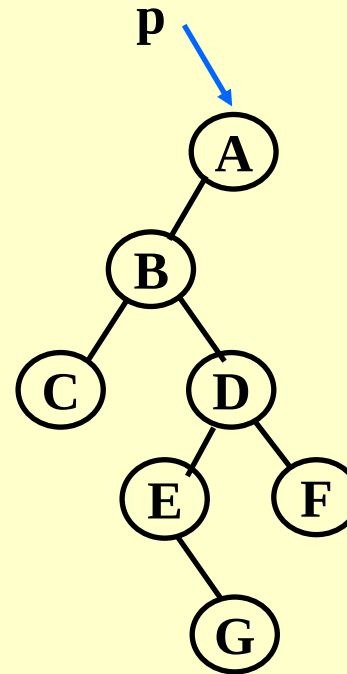
S

访问:

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

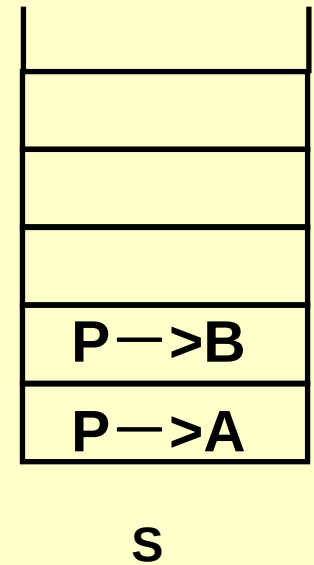
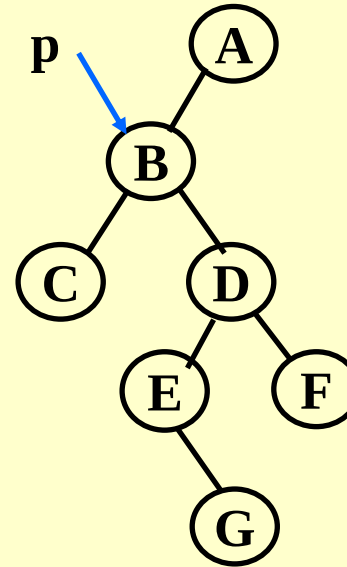


访问:

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

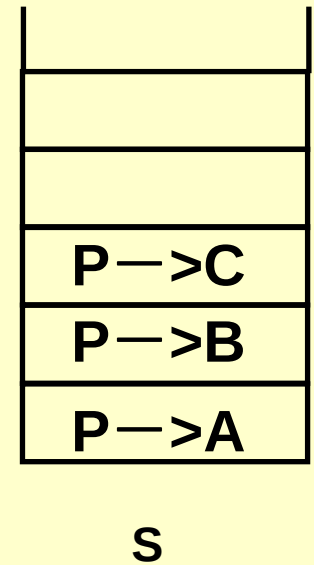
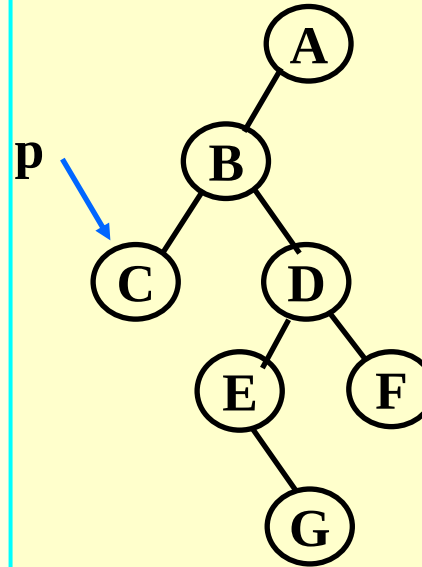


访问:

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```



访问:

```
Status InOrderTraverse(T){
```

```
    InitStack(S); p=T;
```

```
    while (p||StackEmpty(S)){
```

```
        if(p){
```

```
            Push(S,p);p=p->lchild; p
```

```
        }
```

```
    else{
```

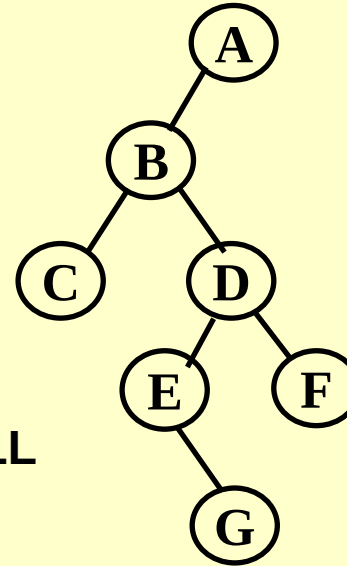
```
        Pop(S,p);
```

```
        printf(p->data);
```

```
        p=p->rchild;
```

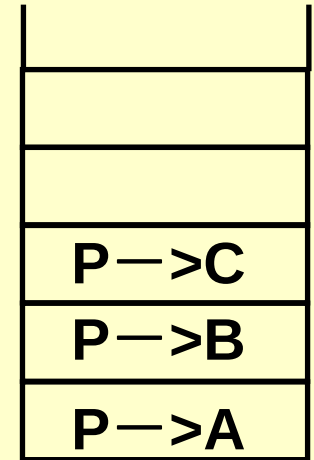
```
    }
```

```
    return OK;}
```



NULL

访问: C

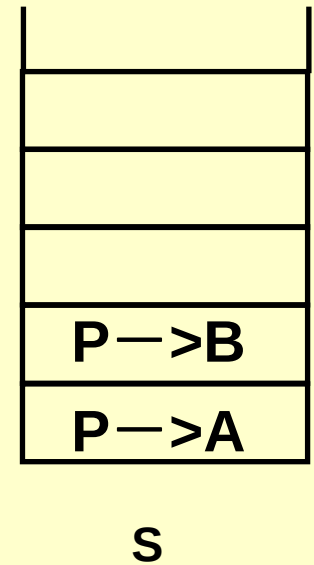
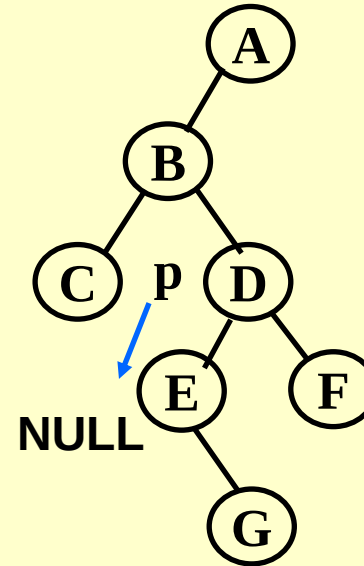


S

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

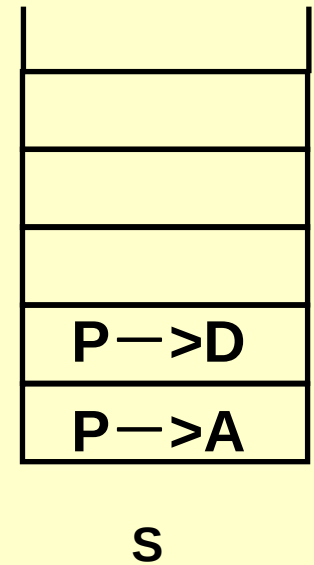
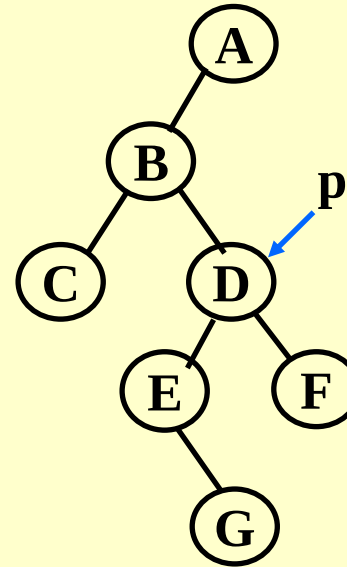


访问: C B


```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

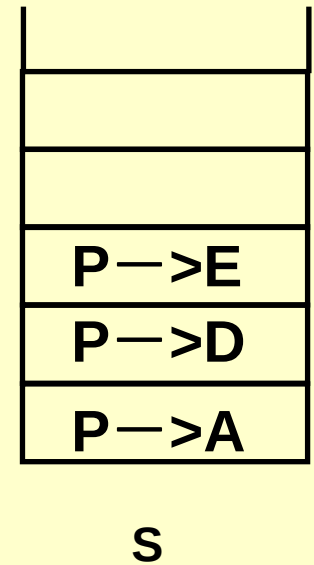
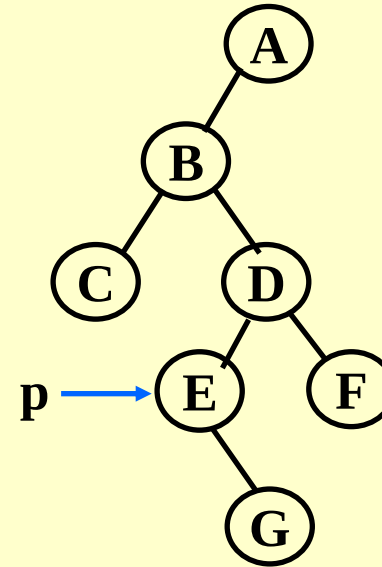


访问： CB

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

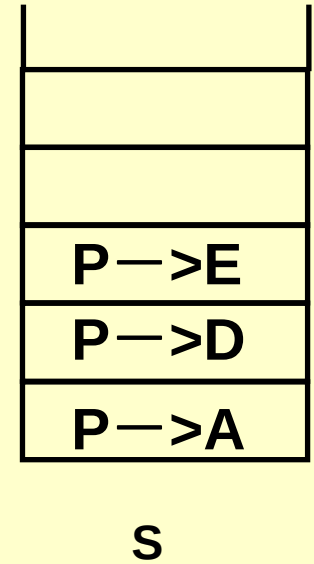
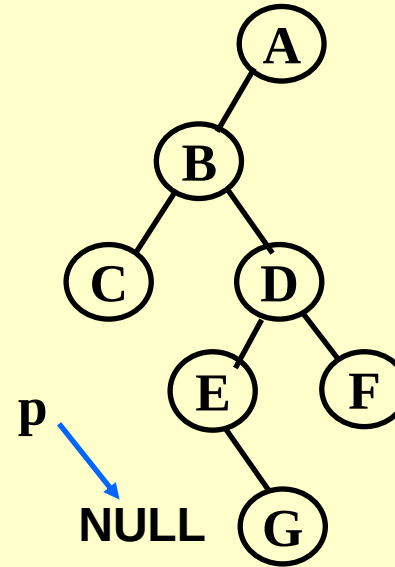


访问： CB

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

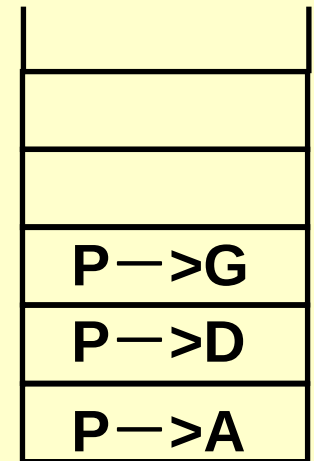
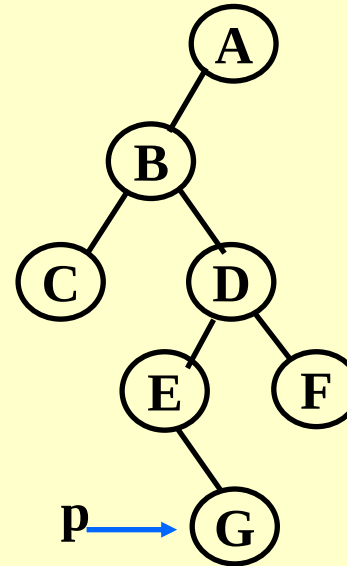


访问： C B E

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```



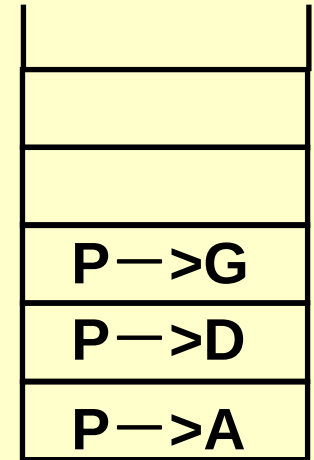
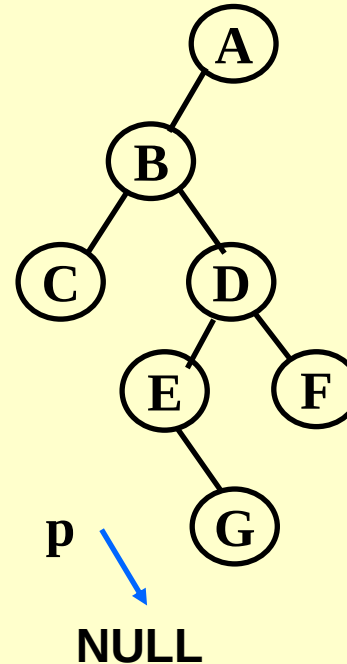
S

访问： CBE

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

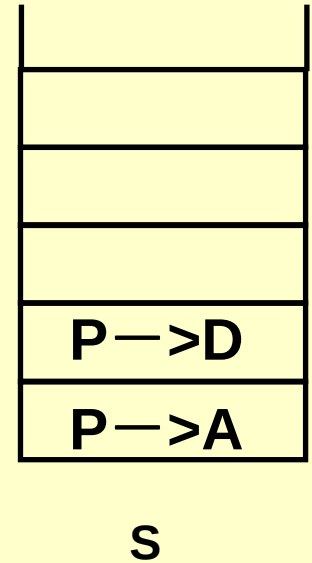
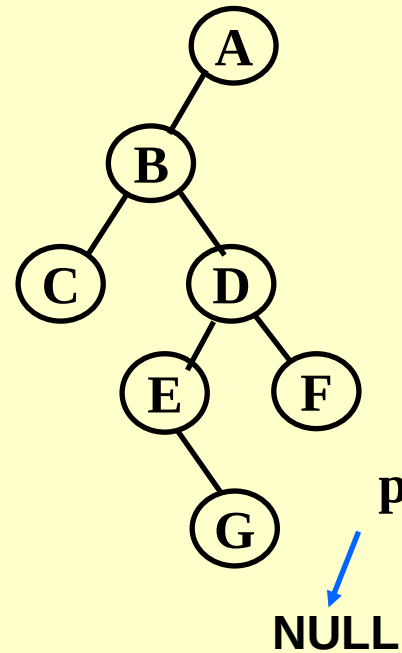


访问: CBE G

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

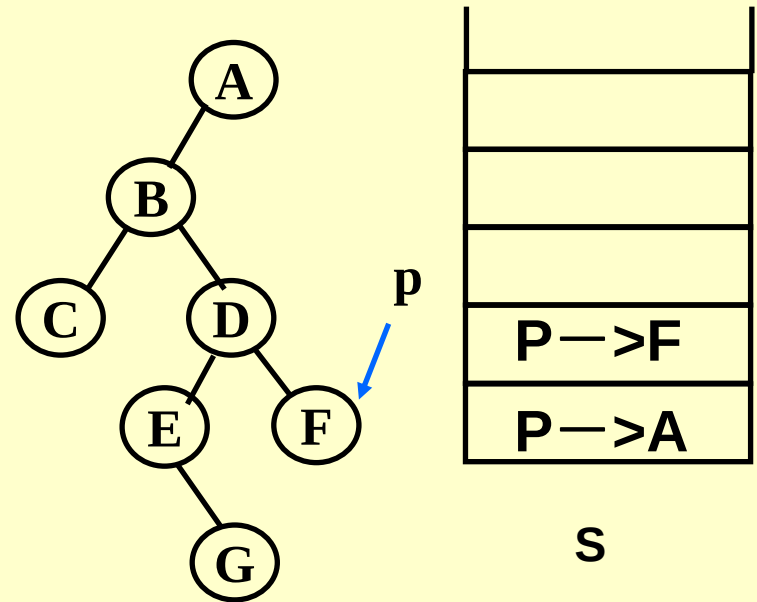


访问: CBEG D

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

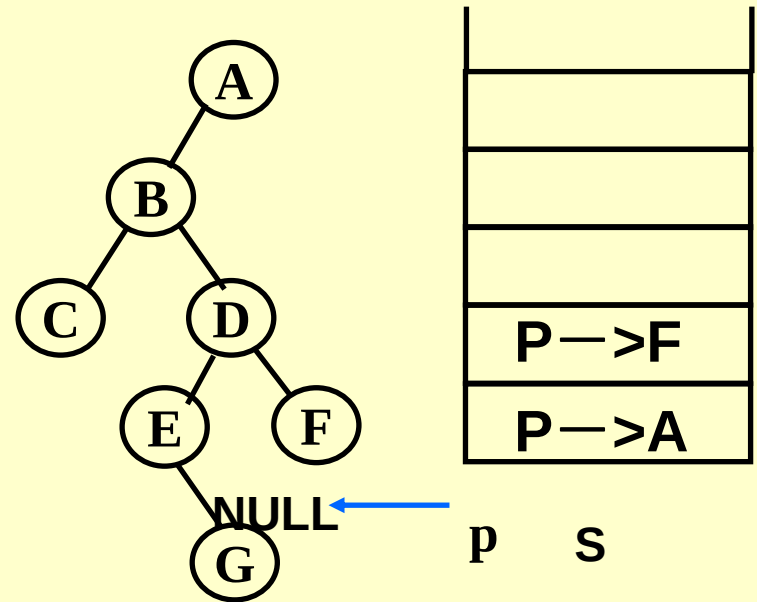


访问: CBEGD

```

Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```

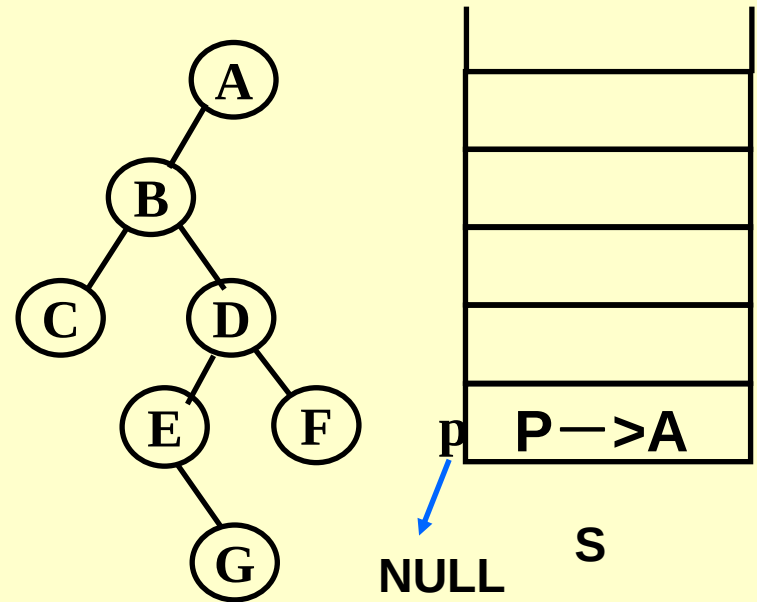


访问: CBEGDF


```

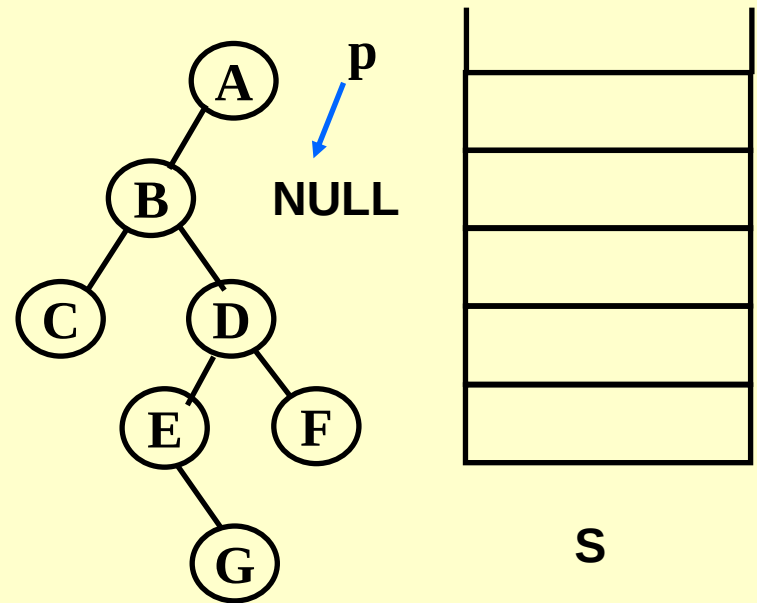
Status InOrderTraverse(T){
    InitStack(S); p=T;
    while (p||StackEmpty(S)){
        if(p){
            Push(S,p);p=p->lchild;
        }
        else{
            Pop(S,p);
            printf(p->data);
            p=p->rchild;
        }
    }
    return OK;}

```



访问: CBEGDFA

```
Status InOrderTraverse(T){  
    InitStack(S); p=T;  
    while (p||StackEmpty(S)){  
        if(p){  
            Push(S,p);p=p->lchild;  
        }  
        else{  
            Pop(S,p);  
            printf(p->data);  
            p=p->rchild;  
        }  
    }  
    return OK;}
```



访问: CBEGDFA

```
Status InOrderNonRecursionTraverse(BiTree T){
```

```
//
```

```
    SqStack S;
```

```
    SElemType p;
```

```
    InitStack(&S);    p=T;
```

```
    while ( _____){
```

```
        if (p){
```

```
            _____; //根指针进栈
```

```
            _____; //指向左子树
```

```
        }
```

```
        else{//根指针退栈，访问根节点，指向右子树
```

```
            _____;
```

```
            _____;
```

```
            _____;
```

```
        }//else
```

```
    }//while
```

```
    DestroyStack(&S);
```

```
    return OK;
```

```
}
```

参照上面的算法填空

非递归 **PreOrder traverse**: **node**, left child, right child;

算法思想:

Push(&S,T);

while S not empty{

Pop p from S;

Visit(p->data);

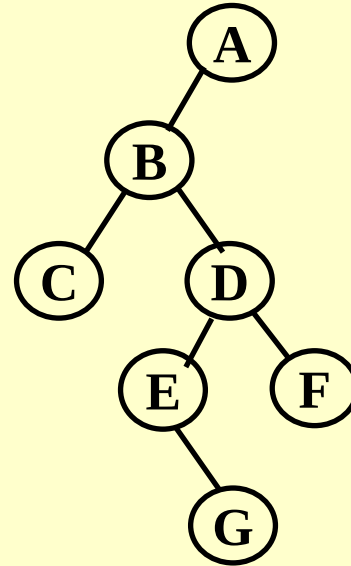
if p->rchild not NULL

Push p->rchild into S;

if p->lchild not NULL

Push p->lchild into S;

//while



用例子演示

```

Status PreOrderNonRecursionTraverse(BiTree T){
    //
    SqStack S;
    SElemType p;
    InitStack(&S);    Push(&S,T); // 根指针入栈
    while(!StackEmpty(S)){
        Pop(&S,&p); //访问根节点
        if(!Visit(p->data)) return ERROR;
        _____ ;
        _____ ;
        _____ ;
        _____ ;
    }//while
    DestroyStack(&S);
    return OK;
}

```

Status PostOrderNonRecursionTraverse(BiTree T){

// 后序遍历二叉树T的非递归算法：向左到尽头，出栈，向右一步再向左到尽头

SqStack S;

SElemType p,q;

InitStack(&S); Push(&S,T); // 根指针入栈

while(!StackEmpty(S)){

while(____&&____) Push(____, ____); //向左走到尽头

GetTop(S,&p);

if(____){Push(____, ____); continue; }//右孩子非空，进栈

if(!StackEmpty(S)){//访问结点，向右一步

Pop(&S,&p);

if(!Visit(p->data)) return ERROR;

while (____&&____&&p==q->rchild){

//若当前为右子树，则继续出栈，用p为q进行比较

Pop(&S,&p);if(!Visit(p->data)) return ERROR;}

GetTop(S,&p);

if(____){Push(____, ____); continue; //右孩子非空，进栈

}else{Pop(&S,&p);if(!Visit(p->data)) return ERROR;}

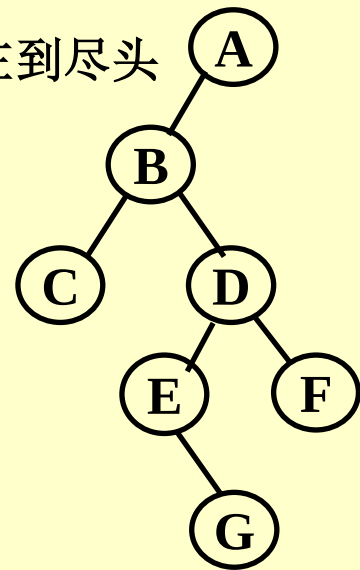
}//if

}//while

DestroyStack(&S);

return OK;

}



演示栈中的元素变化

算法1：从根结点开始，将所有最左结点全部压栈，每当一个结点出栈时，都先扫描该结点的右子树，只有当一个结点的左孩子和右孩子结点均被访问过了，才能访问结点自身。

Status PostOrderNonRecursionTraverse(BiTree T){// 后序遍历二叉树T的非递归算法

SqStack S;

SElemType p,q;

InitStack(&S); Push(&S,T); // 根指针入栈

while(!StackEmpty(S)){

while(GetTop(S,&p)&&p&&(p->lchild||p->rchild)){

Push(&S,p->rchild); // 右子树入栈

Push(&S,p->lchild); // 左子树入栈

}//注意栈中存在空指针，表示某个结点的右子树为空

if(!StackEmpty(S)){//访问结点

Pop(&S,&p);

if (p){

if(!Visit(p->data)) return ERROR;

}else{// 存在右子树为空的结点，继续向上返回

Pop(&S,&p); if(!Visit(p->data)) return ERROR;

}

while (GetTop(S,&q)&&q&&p==q->rchild){//若当前为右子树，则继续出栈

Pop(&S,&p); if(!Visit(p->data)) return ERROR; GetTop(S,&q);

}

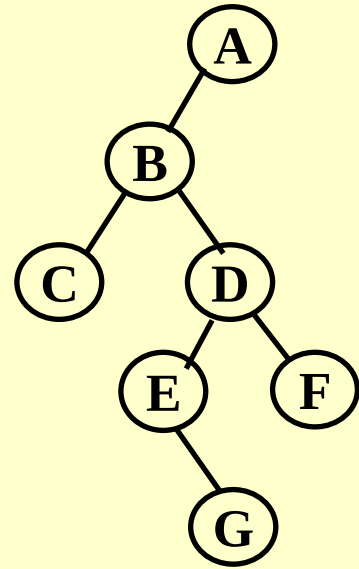
}//if

}//while

DestroyStack(&S);

return OK;

}



算法2:

1. p如果是叶子节点，直接输出

2. p如果有孩子，且孩子没有被访问过，则按照右孩子，左孩子的顺序依次入栈

3. p如果有孩子，而且孩子都已经访问过，则访问p节点

二叉树后序遍历非递归算法特征

当访问某个结点时，栈中所保存的元素正好是该结点的所有祖先。

容易解决下面如下问题：

- (1) 给定一个叶子结点，输出该叶子结点的所有祖先；
- (2) 输出根结点到所有叶子结点的路径；
- (3) 如果二叉树结点的值是数值，求每条路径上值之和。

Experiment 6-3 Non-Recursion Traverse

Problems:

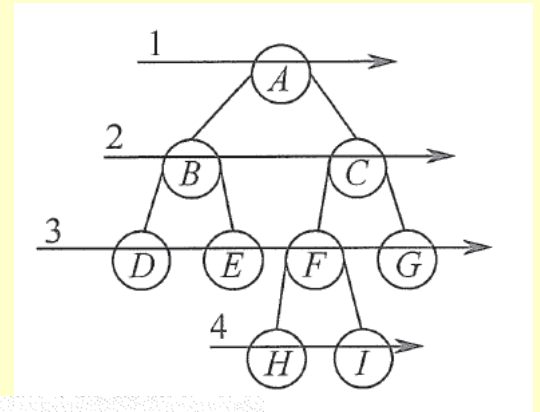
InOrder Non Recursion Traverse

PreOrder Non Recursion Traverse

PostOrder Non Recursion Traverse

扩展：二叉树的层次遍历 力扣102.二叉树的层序遍历

- 先将二叉树根结点入队；
- 出队并访问出队结点，直到队列为空；
 - 若它有左子树，则将左子树根结点入队；
 - 若它有右子树，则将右子树根结点入队；



```
void LevelOrder(BiTree T){
    InitQueue(Q);           //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);           //将根结点入队
    while(!IsEmpty(Q)){     //队列不空则循环
        DeQueue(Q,p)        //队头结点出队
        visit(p);           //访问出队结点
        if(p->lchild!=NULL)
            EnQueue(Q,p->lchild); //左子树不空，则左子树根结点入队
        if(p->rchild!=NULL)
            EnQueue(Q,p->rchild); //右子树不空，则右子树根结点入队
    }
}
```

exp6-3b:扩展6-3算法增加层次遍历 如何记录每一层的信息？

扩展：二叉树的层次遍历

```
vector<vector<int>> levelOrder(TreeNode* root) {
```

```
    queue<TreeNode*> que;
```

```
    if (root != NULL) que.push(root);
```

```
    vector<vector<int>> result;
```

```
    while (!que.empty()) {
```

```
        int size = que.size();
```

```
        vector<int> vec;
```

```
        // 这里一定要使用固定大小size，不要使用que.size()
```

```
        for (int i = 0; i < size; i++) { //处理一层
```

```
            TreeNode* node = que.front();
```

```
            que.pop();
```

```
            vec.push_back(node->val);
```

```
            if (node->left) que.push(node->left);
```

```
            if (node->right) que.push(node->right);
```

```
        }
```

```
        result.push_back(vec);
```

```
    }
```

```
    return result;
```

```
}
```

```
struct TreeNode {
```

```
    int val;
```

```
    TreeNode *left;
```

```
    TreeNode *right;
```

```
    TreeNode(int x) : val(x),
```

```
    left(NULL), right(NULL) {}
```

```
};
```

扩展：二叉树的层次遍历

- 102.二叉树的层序遍历
- 107.二叉树的层次遍历II(返回其节点值自底向上的层次遍历)

反转结果数组

- 199.二叉树的右视图(返回从右侧所能看到的节点值)

if (i == (size - 1)) result.push_back(node->val); // 将每一层的最后元素放入result数组中

- 637.二叉树的层平均值(每一层的平均值)

sum += node->val;

result.push_back(sum / size); // 将每一层均值放进结果集

- 429.N叉树的层序遍历
- 515.在每个树行中找最大值
- 116.填充每个节点的下一个右侧节点指针(完全二叉树层节点)
- 117.填充每个节点的下一个右侧节点指针II
- 104.二叉树的最大深度
- 111.二叉树的最小深度
- 662. 二叉树的宽度(最大)

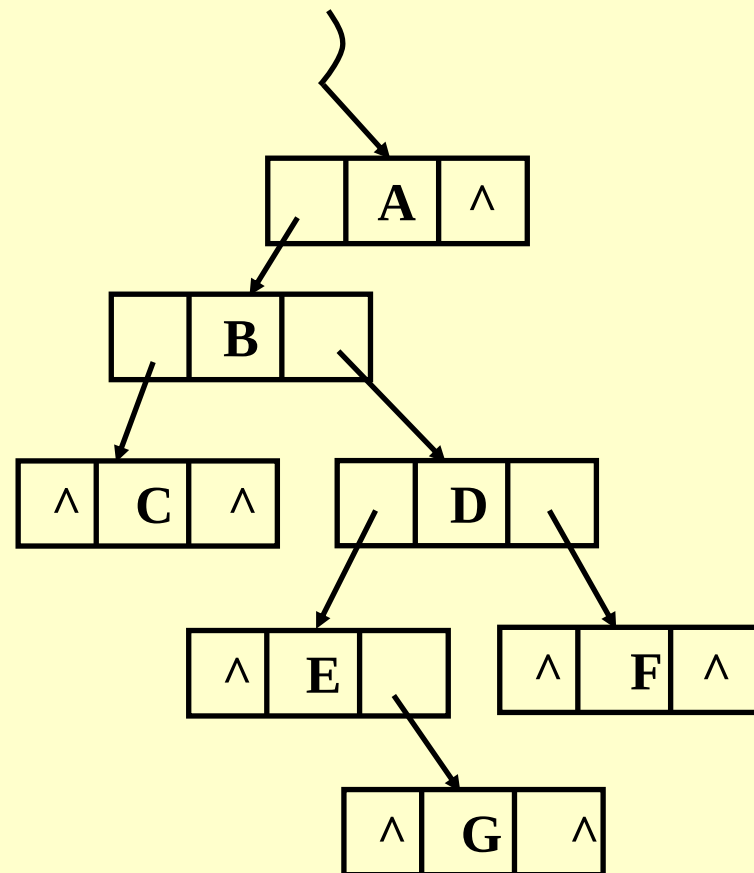
Experiment 6-3b level Order Traverse Binary Tree

Problems:

```
cout << "levelOrder Traverse Binary Tree" << endl;
cout << "===== " << endl;
cout << "1. Create a binary tree; 2. levelOrder;" << endl;
cout << "3. levelOrderBottom; 4. rightSideView;" << endl;
cout << "5. averageOfLevels; 6. connectNext;" << endl;
cout << "7. widthOfBinaryTree; 0. Exit;" << endl;
cout << "===== " << endl;
```

§6.3.3 线索二叉树:

1、引入原因: 当以二叉链表作为存储结构时,只能找到结点的左右孩子的信息,而不能在结点的任一序列的**前驱与后继**信息,这种信息只有在遍历的动态过程中才能得到。利用空指针存储前驱后继信息。



2、几个概念

前驱与后继: 在二叉树的先序、中序或后序遍历序列中两个相邻的结点互称为前驱与后继

线索: 指向前驱或后继结点的指针

线索二叉树: 加上线索的二叉链表表示的二叉树

线索化: 对二叉树按某种遍历次序使其变为线索二叉树的过程

3、手段：增加标志域。

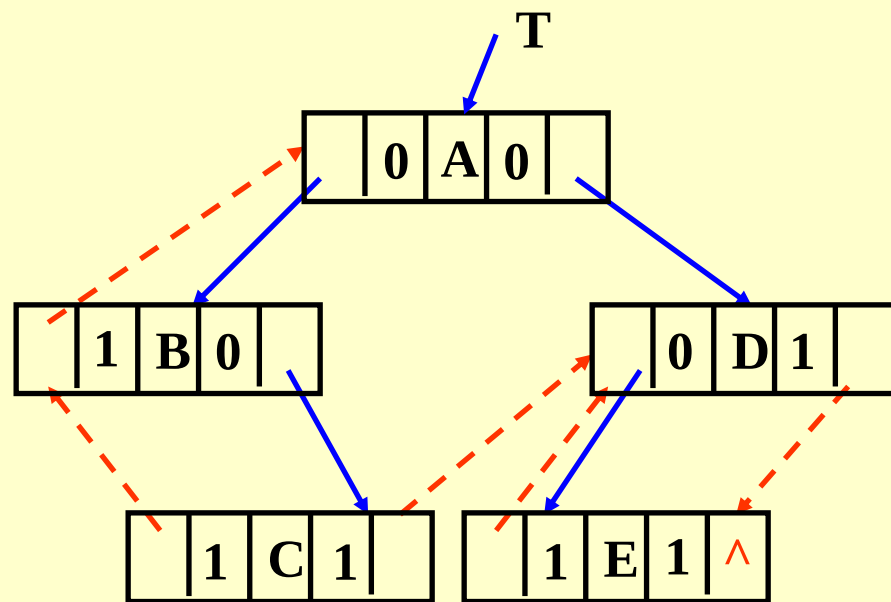
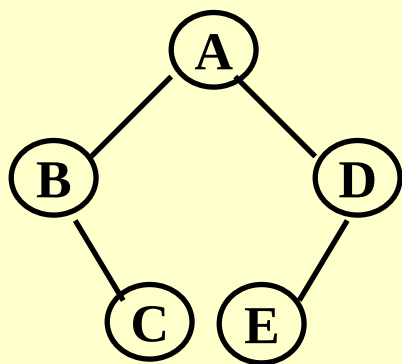
lchild	ltag	data	rtag	rchild
--------	------	------	------	--------

其中:

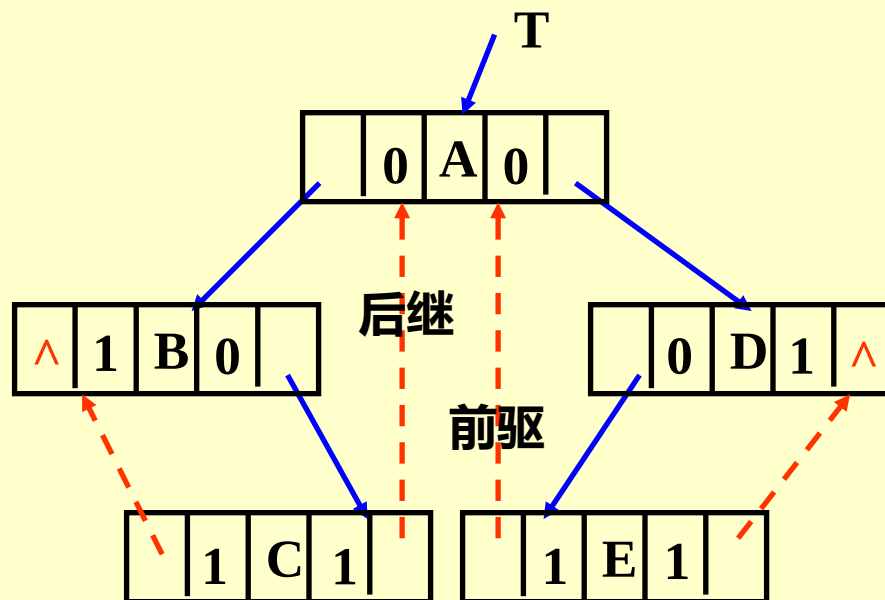
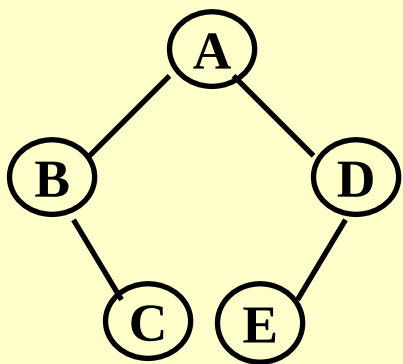
$$\begin{aligned} \text{ltag} &= \begin{cases} 0 & \text{lchild 域指示结点的左孩子} \\ 1 & \text{lchild 域指示结点的前驱} \end{cases} \\ \text{rtag} &= \begin{cases} 0 & \text{rchild 域指示结点的右孩子} \\ 1 & \text{rchild 域指示结点的后继} \end{cases} \end{aligned}$$

二叉树的二叉线索存储表示:

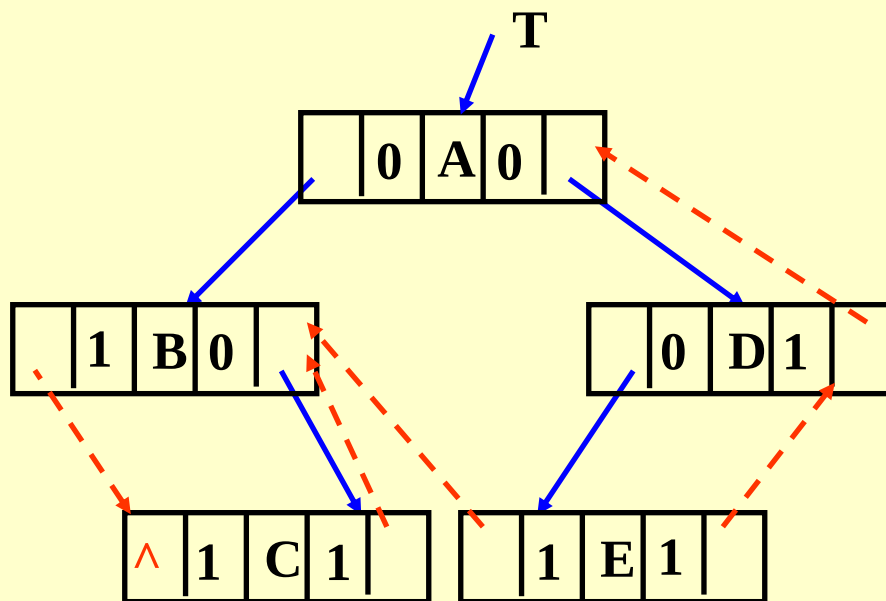
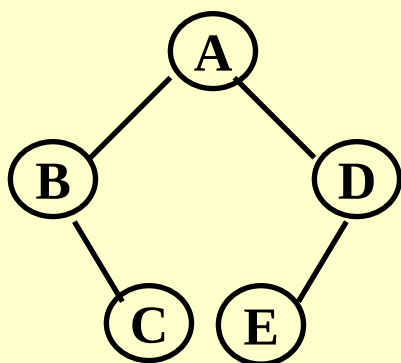
```
typedef enum{Link,Thread} PointerTag;
//Link= =0:指针,Thread= =1:线索
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild,*rchild;
    PointerTag ltag, rtag;
}BiTNode, *BiTree;
```



先序序列: ABCDE
先序线索二叉树

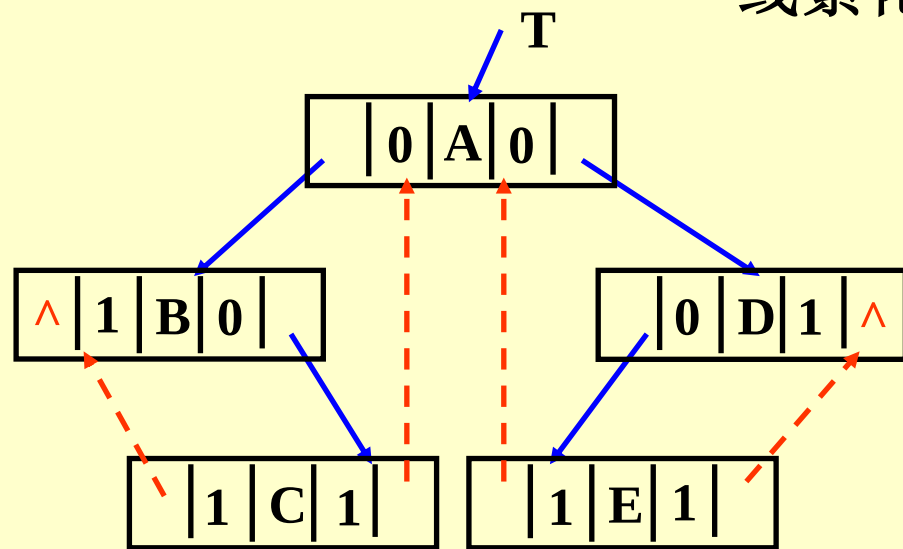


中序序列: **BCAED**
 中序线索二叉树

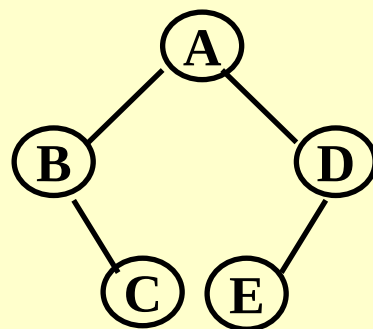


后序序列: **CBEDA**
 后序线索二叉树

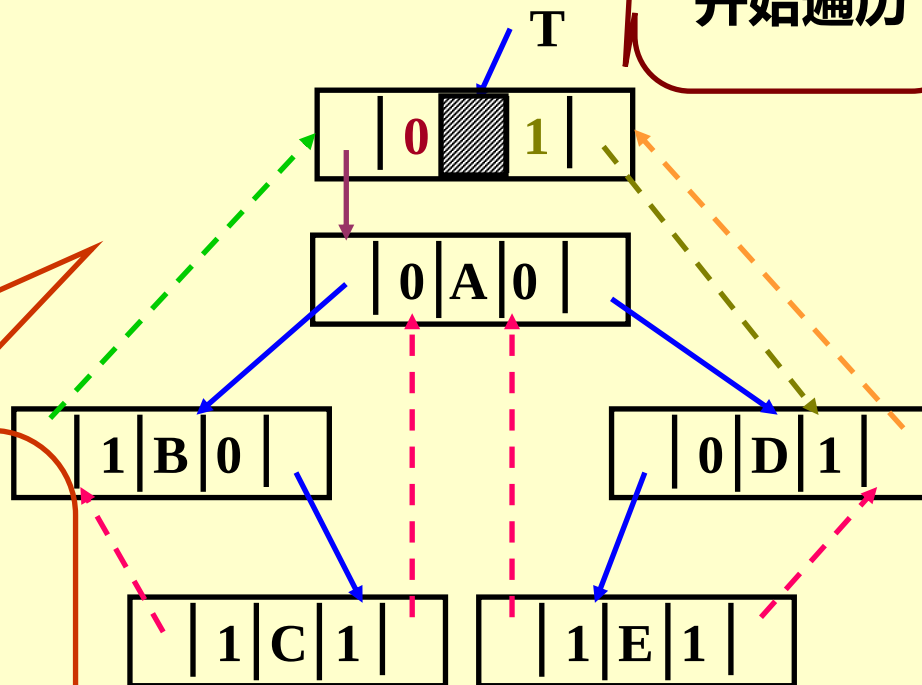
线索化的二叉树像一个双向循环链表



中序序列: **BCAED**
中序线索二叉树



这样即可以
从第一个节点也
可从最后一个节点
开始遍历



中序序列: **BCAED**
带头结点的中序线索二叉树

为了方便, 增加头结点:
ltag=0, lchild指向根结点
rtag=1, rchild指向遍历序列中最后
一个结点
遍历序列中第一个结点的lchild域和
最后一个结点的rchild域都指向头结
点

基本思想：向左到头，输出；右孩子处理，一直利用后继指针。

```
Status InorderTraverse_Thr(T){//不可运行
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

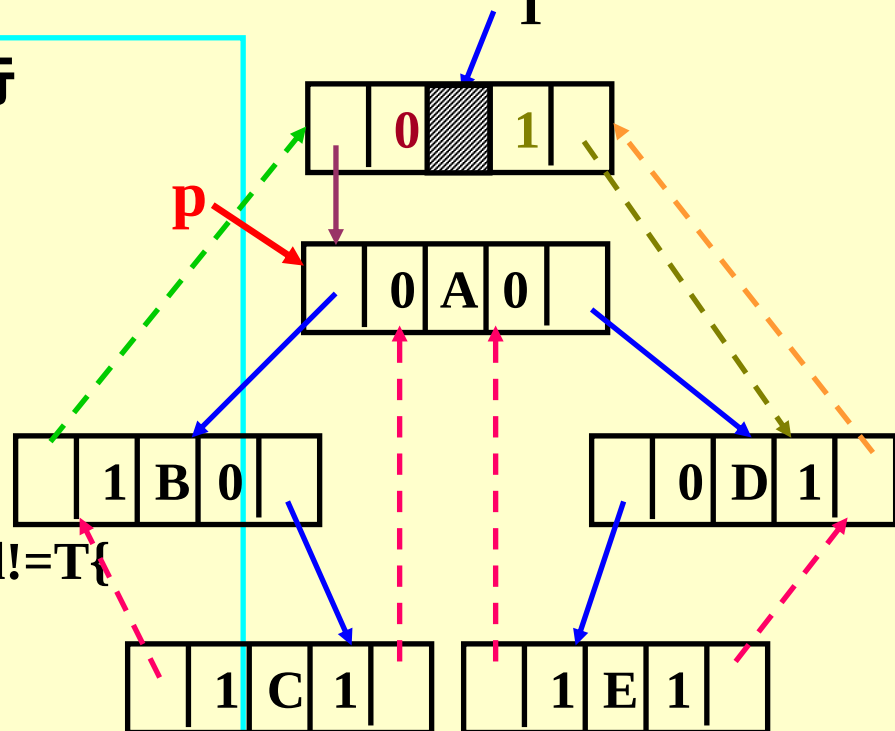
```
        }//无左孩子的右分支回退过程
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出：

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

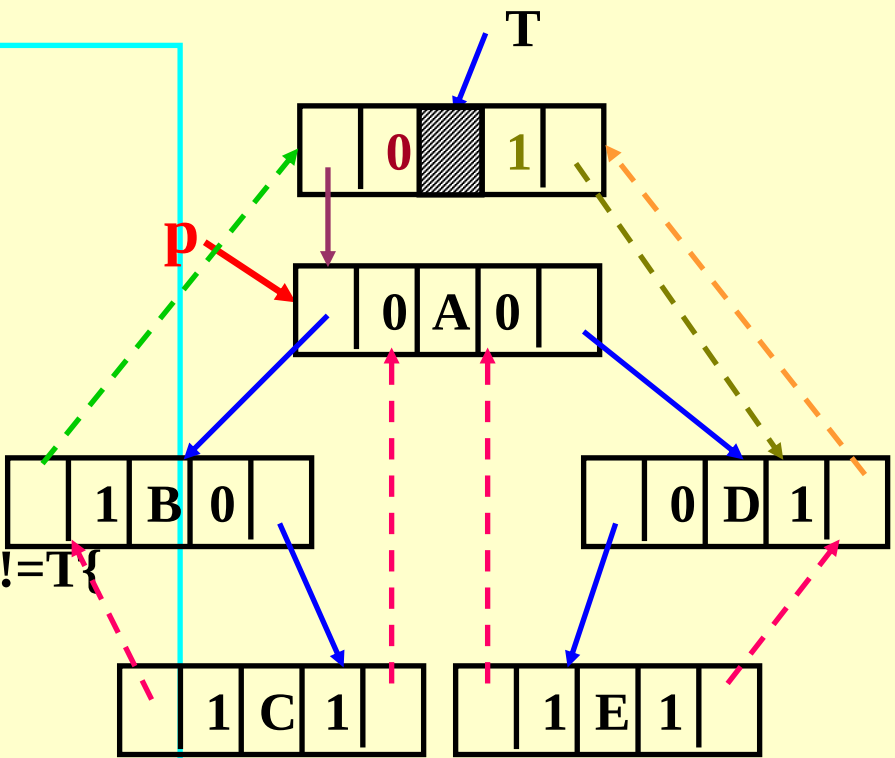
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出:

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild; p
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

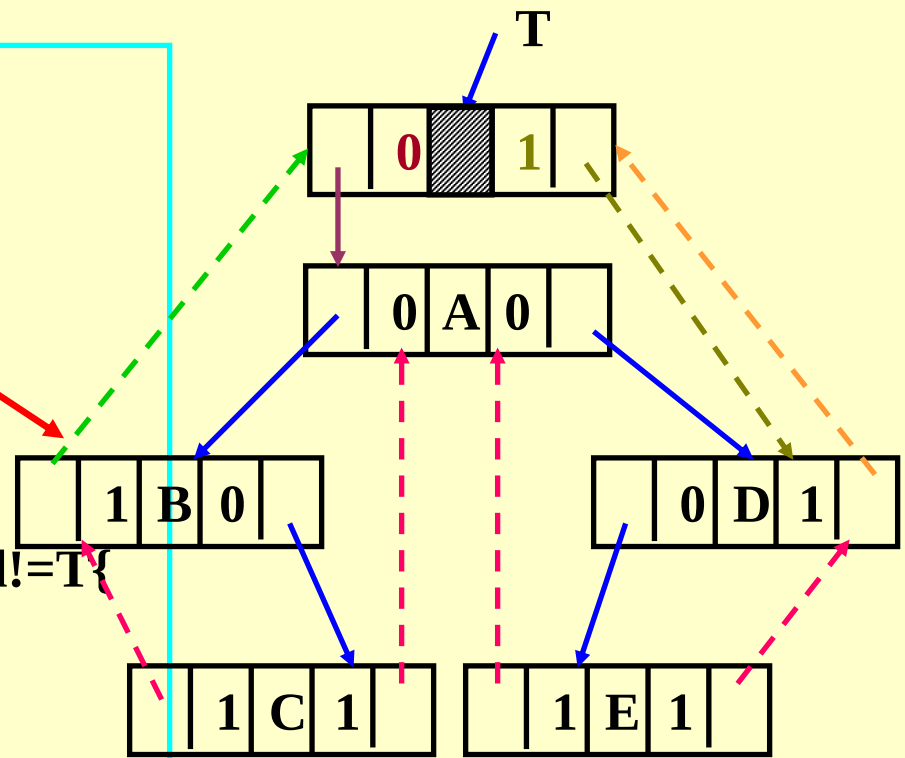
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出：

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild; p
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T){
```

```
            p=p->rchild; printf(p->data);
```

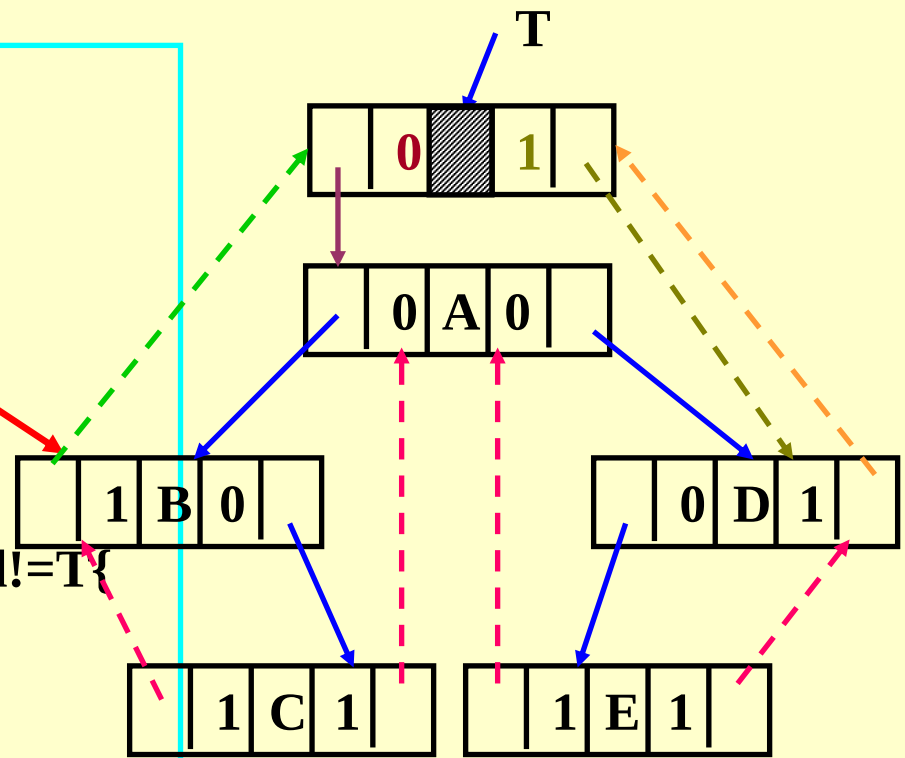
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出： B

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild; p
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

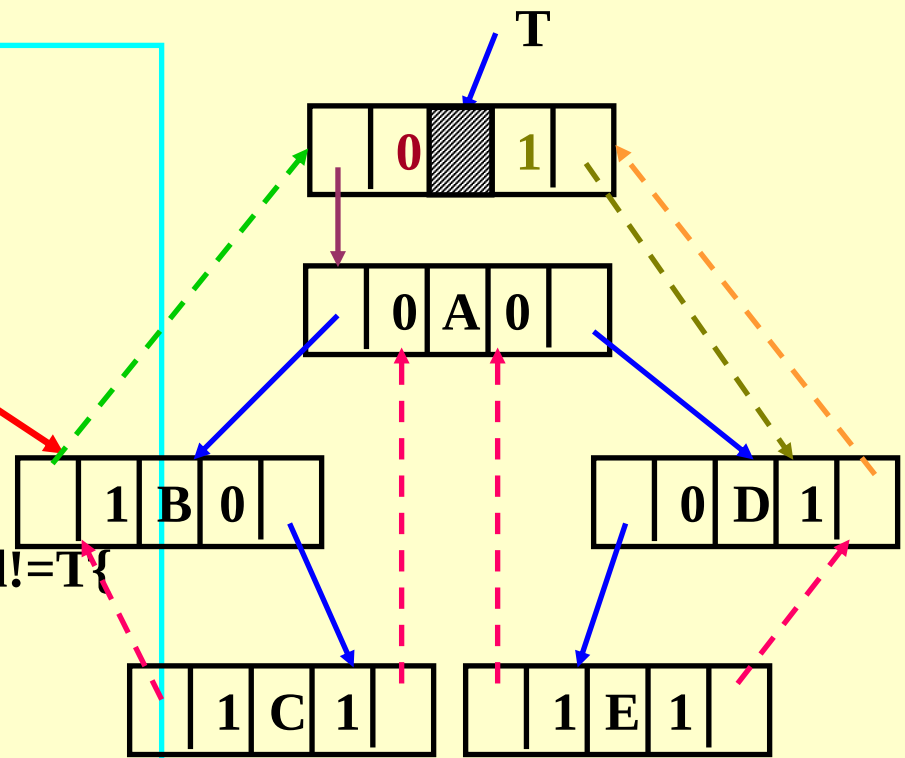
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出： B


```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

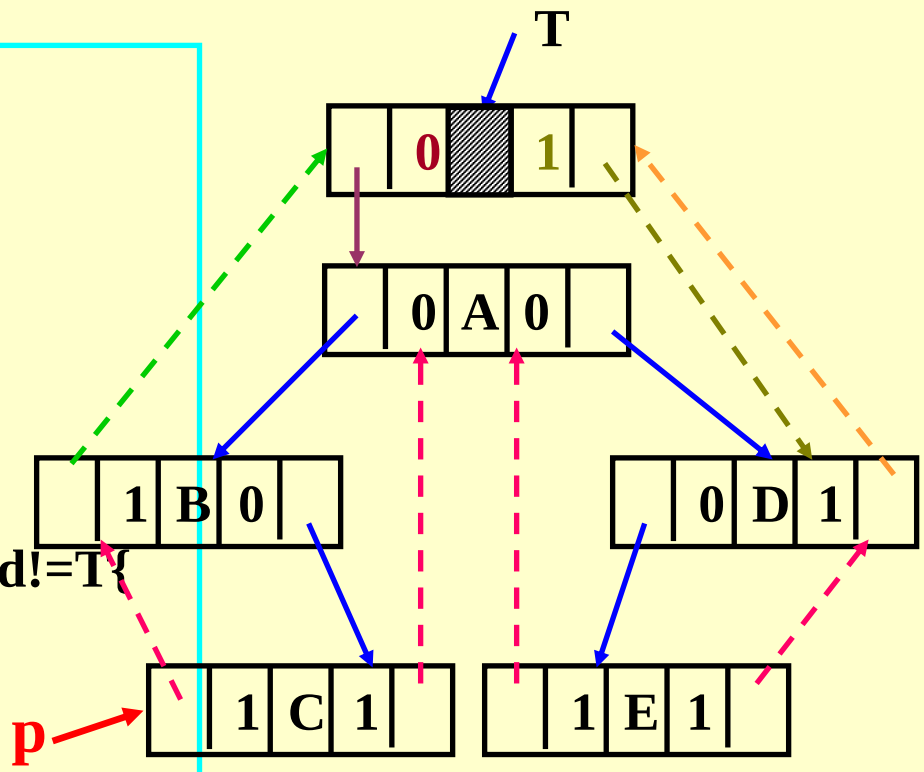
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出： B

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

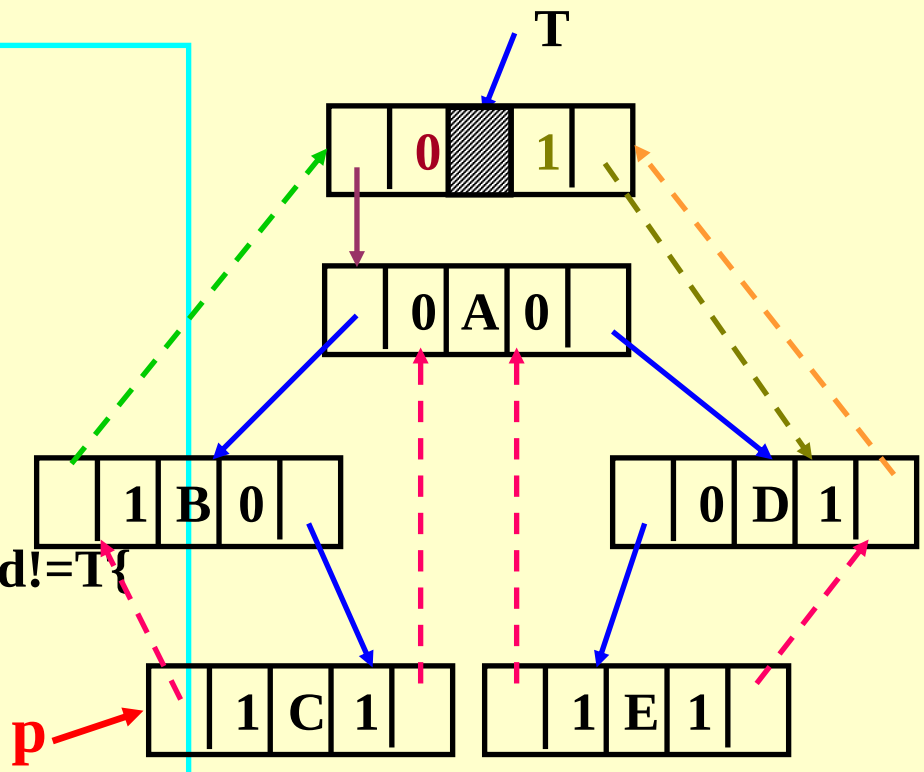
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

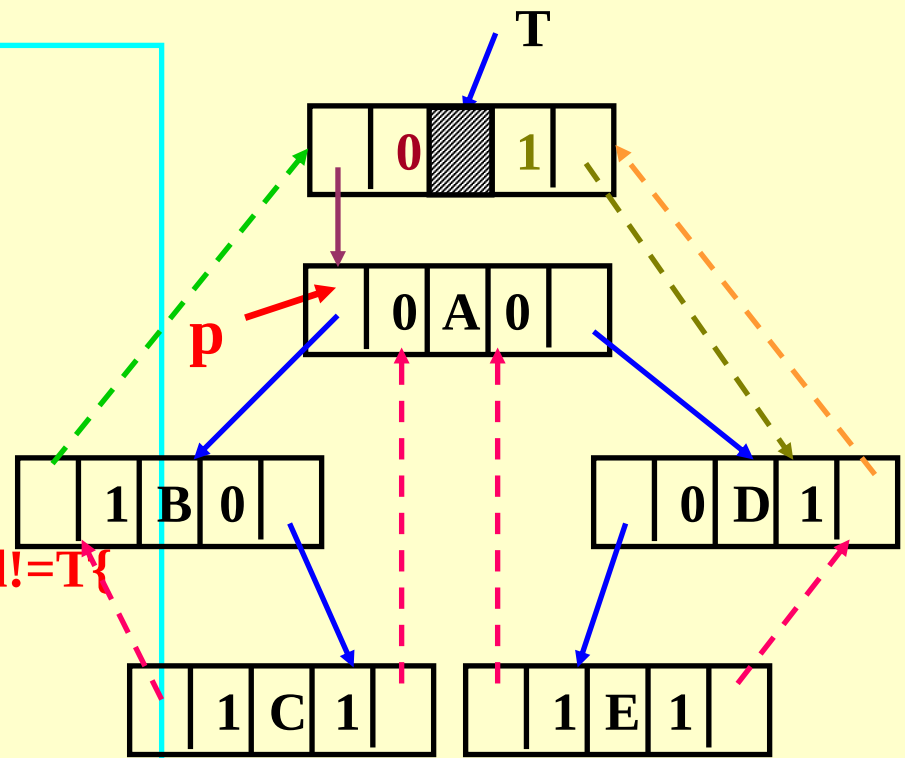
```
}
```



带头结点的中序线索二叉树

输出：BC

```
Status InorderTraverse_Thr(T){  
    p=T->lchild;  
    while(p!=T){  
        while(p->LTag==Link) p=p->lchild;  
        printf(p->data);  
        while(p->RTag==Thread && p->rchild!=T){  
            p=p->rchild; printf(p->data);  
        }  
        p=p->rchild;  
    }  
    return OK;  
}
```



带头结点的中序线索二叉树

输出：BCA

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T){
```

```
            p=p->rchild; printf(p->data);
```

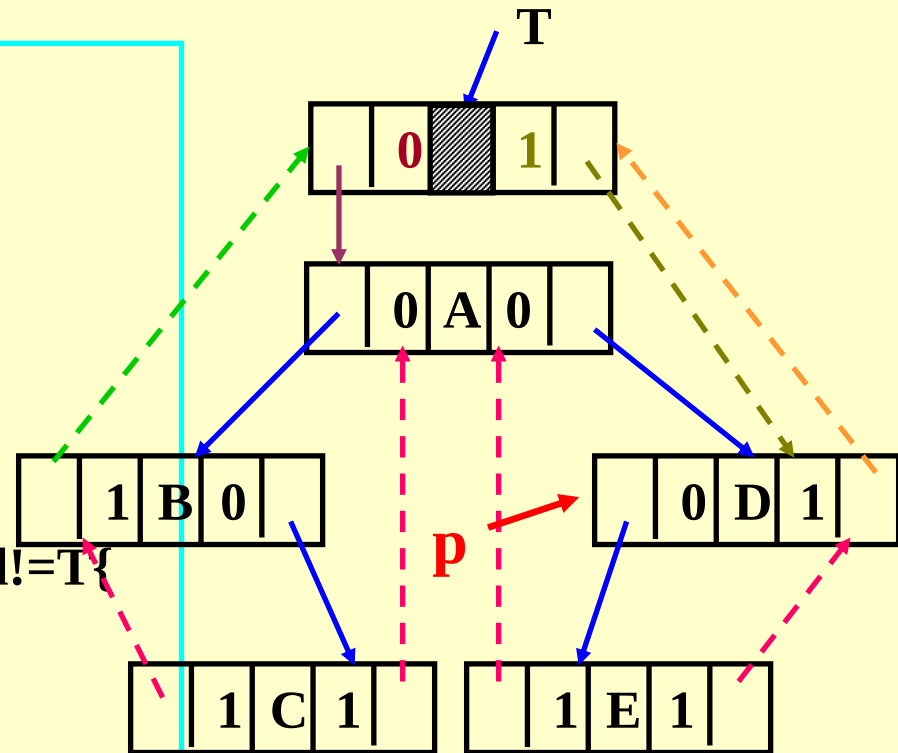
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出：BCA

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T{
```

```
            p=p->rchild; printf(p->data);
```

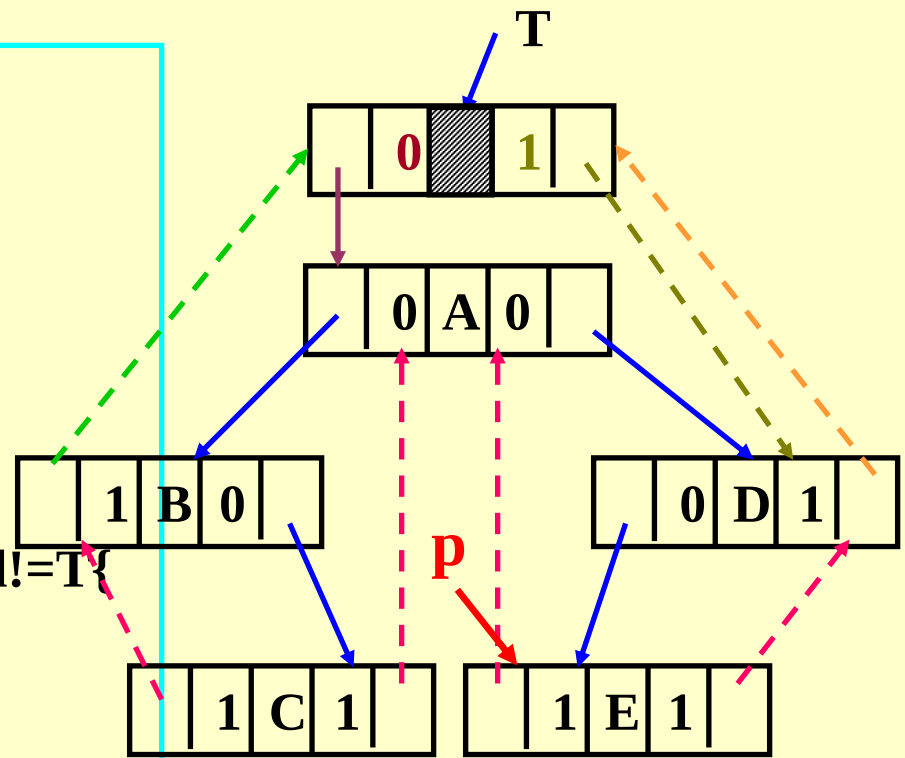
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出：BCA

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T){
```

```
            p=p->rchild; printf(p->data);
```

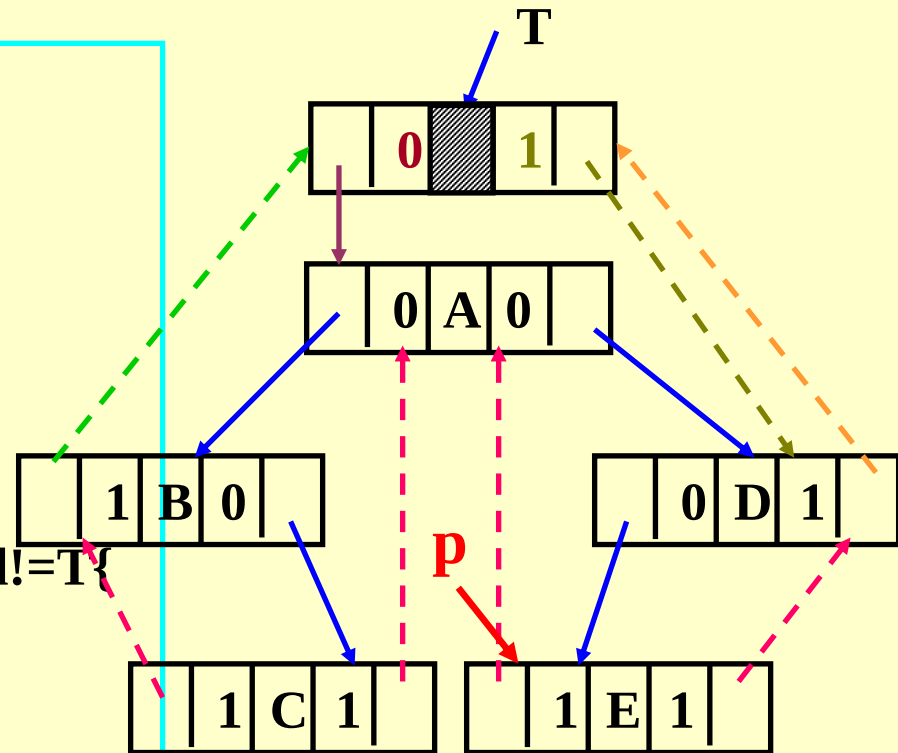
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

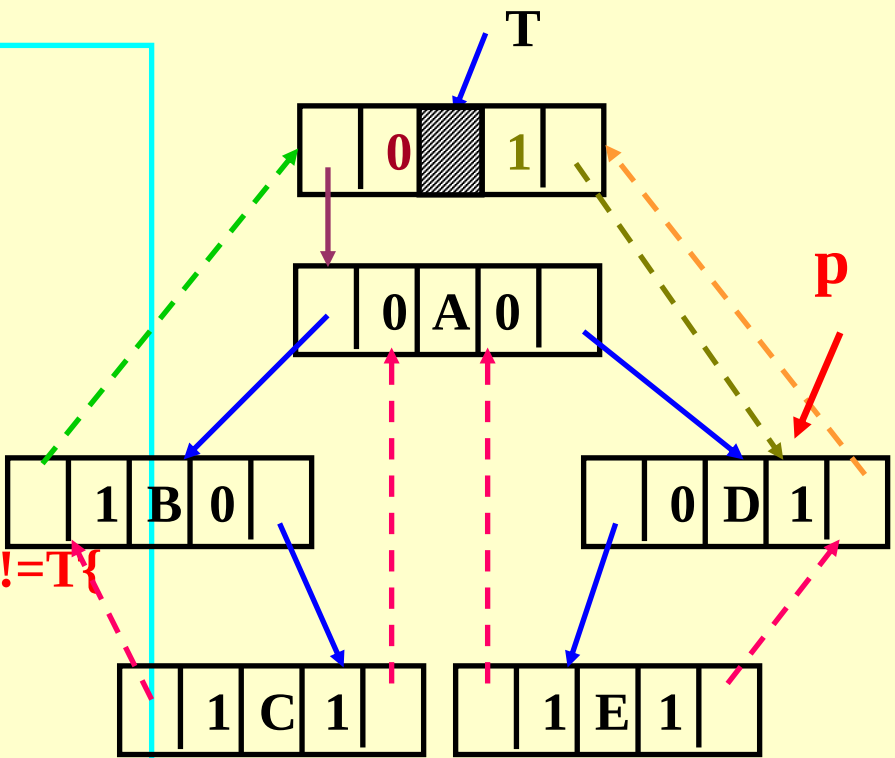
```
}
```



带头结点的中序线索二叉树

输出：BCAE

```
Status InorderTraverse_Thr(T){  
    p=T->lchild;  
    while(p!=T){  
        while(p->LTag==Link) p=p->lchild;  
        printf(p->data);  
        while(p->RTag==Thread && p->rchild!=T){  
            p=p->rchild; printf(p->data);  
        }  
        p=p->rchild;  
    }  
    return OK;  
}
```



带头结点的中序线索二叉树

输出：BCAED

➤ 遍历算法不用递归，不用栈。

```
Status InorderTraverse_Thr(T){
```

```
    p=T->lchild;
```

```
    while(p!=T){
```

```
        while(p->LTag==Link) p=p->lchild;
```

```
        printf(p->data);
```

```
        while(p->RTag==Thread && p->rchild!=T){
```

```
            p=p->rchild; printf(p->data);
```

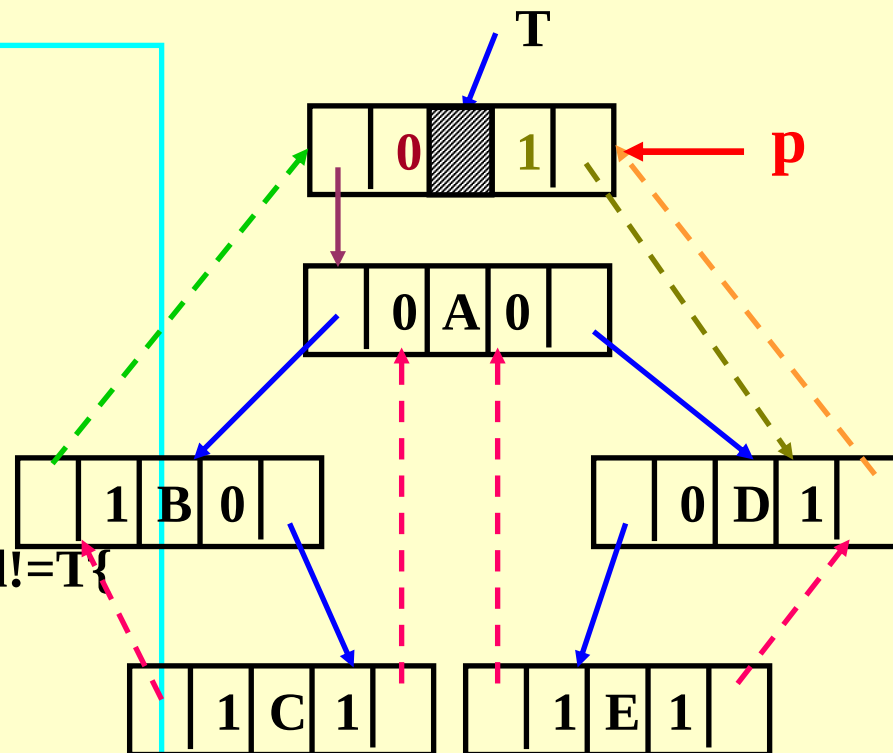
```
        }
```

```
        p=p->rchild;
```

```
    }
```

```
    return OK;
```

```
}
```



带头结点的中序线索二叉树

输出：BCAED

void InOrderTraverse(BiTree T) //可运行

{//遍历中序线索二叉树

BiTree p;

p = T->lchild; //p指向根

while (_____) //当p不为头结点时

{//线索化的二叉树就像一个双向循环链表

while (p->ltag == _____)

_____ ; //向左到底

VistTree(p);

while (p->rtag == _____ && p->rchild != T)

{

p = p->rchild; VistTree(p);

}

//若p的右指针为线索则p->rchild为p的后继

//因此可以直接令p=p->rchild然后访问

p = _____ ;

//后继访问完毕则遍历右子树

}

}

BiTree InOrderThreading(BiTree Thr, BiTree T) //可运行

{//线索化二叉树

Thr = (BiTree)malloc(sizeof(BiTNode));//建立头

if (!Thr) exit(1);

Thr->ltag = link;

Thr->rchild = Thr;

Thr->rtag = thread;

if (T == NULL) { Thr->lchild = Thr; }

else

{

Thr->lchild = T; //令头结点的左指针指向根结点

pre = Thr;

InThreading(T); //中序线索化核心过程

pre->rchild = Thr;

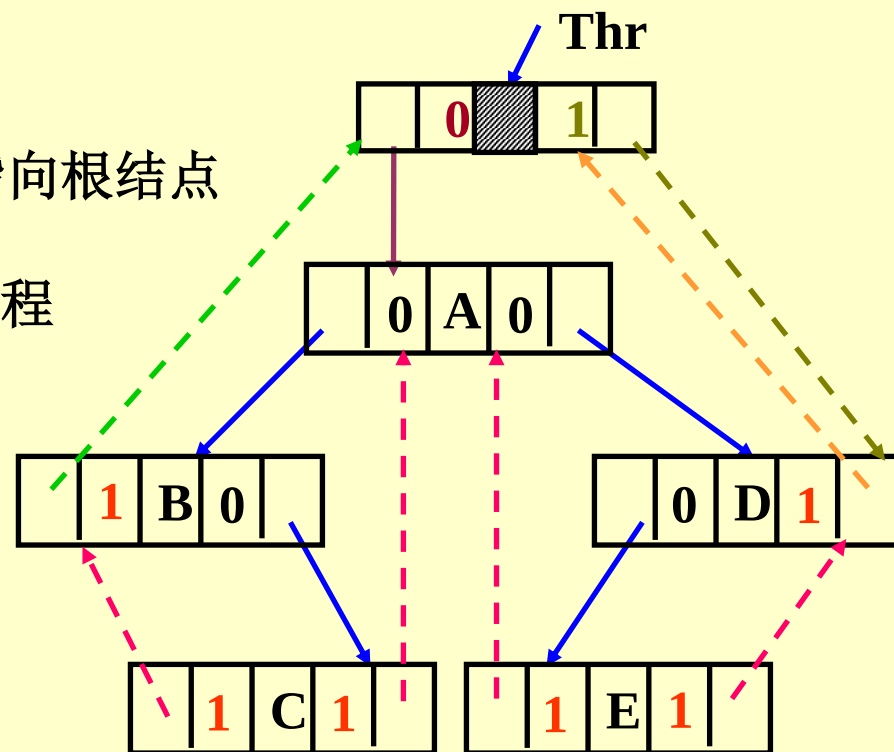
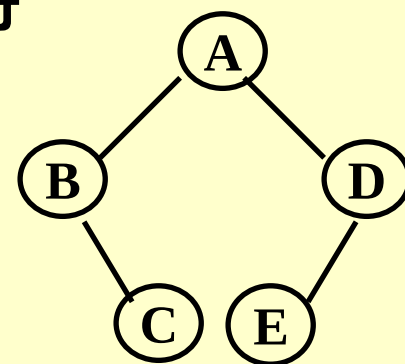
pre->rtag = thread;

Thr->rchild = pre;

}

return Thr;

}



pre为全局变量指p的前驱结点

```
void InThreading(BiTree p) //可运行
```

```
{//中序线索化核心过程
```

```
    if (p != NULL)
```

```
    {
```

```
        InThreading(_____); //递归线索左子树
```

```
        if (p->lchild == NULL) {
```

```
            p->lchild = _____; //指向前驱
```

```
            p->ltag = _____;
```

```
        }
```

```
        if (pre->rchild == NULL) {
```

```
            pre->rchild = _____; //指向后继
```

```
            pre->rtag = _____;
```

```
        }
```

```
        pre = p;
```

```
        InThreading(_____); //递归线索右子树
```

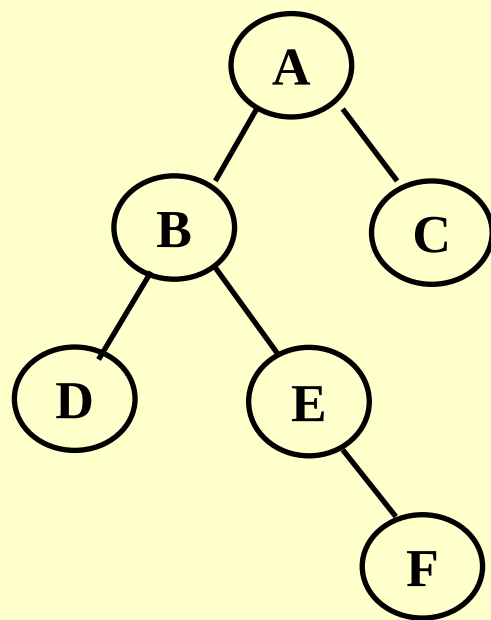
```
    }
```

```
}
```

pre为全局变量指p的前驱结点

练习

构建三种线索二叉树



Experiment 6-4 Thread Binary Tree

Problems:

void CreatBinaryTree(BiTree &T)

void InOrderThreading(BiTree &Thr,BiTree &T)

void InOrderTraverse(BiTree &T)

1. 【2017统考真题】要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的条件是（ ）。
- A. 只有左子树 B. 只有右子树 C. 结点的度均为1 D. 结点的度均为2
2. n 个结点的线索二叉树上含有的线索为（ ）。
- A. $2n$ B. $n-1$ C. $n+1$ D. n
3. 判断线索二叉树中 $*p$ 结点有右孩子结点的条件是（ ）。
- A. $p \neq \text{NULL}$ B. $p \rightarrow \text{rchild} \neq \text{NULL}$ C. $p \rightarrow \text{rtag} == 0$ D. $p \rightarrow \text{rtag} == 1$
4. 一棵左子树为空的二叉树在先序线索化后，其中空的链域的个数是（ ）。
- A. 不确定 B. 0 C. 1 D. 2
5. 若 X 是二叉中序线索树中一个有左孩子的结点，且 X 不为根，则 X 的前驱为（ ）。
- A. X 的双亲 B. X 的右子树中最左的结点
C. X 的左子树中最右结点 D. X 的左子树中最右叶结点
6. 【2013统考真题】若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y ，则 X 的右线索指向的是（ ）。
- A. X 的父结点 B. 以 Y 为根的子树的最左下结点
C. X 的左兄弟结点 D. 以 Y 为根的子树的最右下结点
- 7*. 【2015统考真题】先序序列为 a,b,c,d 的不同二叉树的个数是（ ）。
- A. 13 B. 14 C. 15 D. 16
8. 下列序列中，不能唯一地确定一棵二叉树的是（ ）。
- A. 层次序列和中序序列 B. 先序序列和中序序列
C. 后序序列和中序序列 D. 先序序列和后序序列